The Living Map: Spatial Memory for Emergency Robots

TSYP14 Technical Challenge — IEEE RAS × IEEE AESS Tunisia — Phase 1 Technical Report

Team: **The Architects** · Repository: `https://github.com/housseinelyatim-exe/the-living-map` · Demo video: `media/demo.mp4`

Abstract. When a robot fails inside a collapsed or burning building, what it learned usually dies with it. We give the building its own memory instead. An autonomous *Writer* robot explores a GPS-denied **fire/hazardous building**. It detects three event types (**fire, victims, blocked passages**) and leaves 16-byte BLE beacons that state *what* was found, *where to go* and *when* it was written. It carries its log out past the radio boundary to an *Outside Network Area* (ONA). The ONA *receives* the log, *translates* private coordinates to GPS using a new gyro-drift loop closure, *carries* the map over LoRa to a distant command post, and *briefs* an *Executor* robot. The Executor enters with no map of its own, navigates beacon-to-beacon and rescues the victim the commander chose. A full Python simulation runs the whole chain end to end, with six injected failures. All six missions succeed, and 21 automated tests pass.

1 System Architecture and Data Flow

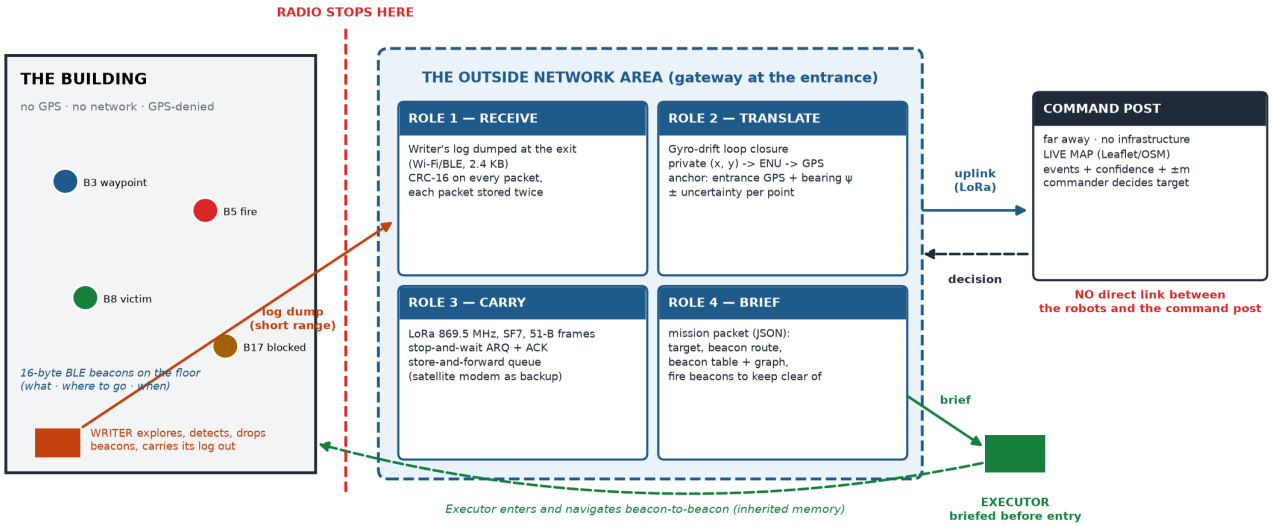


Figure 1: Architecture. All inside↔outside traffic goes through the ONA; robots never talk to the command post (this is checked by `test_no_direct_robot_to_command_post_link`).

Mission sequence (Fig. **??**; a sequence diagram is in `docs/dataflow.png`): (1) The Writer drops the exit beacon B0 at the entrance and explores. (2) On events, at doorways and every 3 m, it drops a beacon. (3) When its battery or the frontier runs out, it returns and dumps a binary log at the entrance. (4) The ONA checks CRCs, estimates odometry drift, converts everything to GPS and builds a beacon graph. (5) LoRa frames update the command post's live map; the commander picks a victim; the decision returns to the ONA. (6) The ONA briefs the Executor with a route over the beacon graph. (7) The Executor navigates using the beacons, confirms the victim and returns. (8) Its report goes back through the ONA to the map.

2 Writer Robot Autonomy

The Writer builds an occupancy grid (0.5 m cells) from a 3 m, 72-ray lidar. It runs **frontier exploration**: it repeatedly drives (A*, 8-connected, no corner cutting) to the nearest known-free cell that borders unknown space. Decisions:

- **Heat no-go zones:** cells localised as burning by the thermal sensor, inflated by one cell, are never entered.
- **Battery-aware return:** at each step the robot plans the path home. When battery $< 1.3 \times$ cost-to-exit $+ 15$, it turns back. Coming back is what saves the data and gives the loop closure (§5).
- **Beacon budget:** 24 beacons; the last 4 are reserved for events.

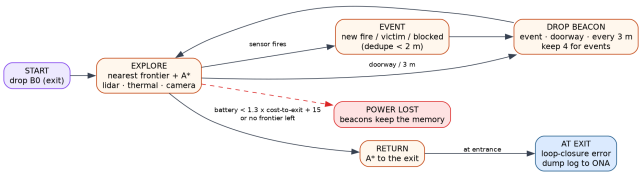


Figure 2: Writer decision state machine.

3 Event Detection and Beacon Deposition

Event	Detection (simulated sensor → Phase 2 part)	Severity byte
Fire	Air temperature $T = 25 + \sum 400e^{-d/0.6}$ °C (line of sight) > 60 °C, then hot cells localised in view → MLX90614 / AMG8833	$2(T - 25)$
Victim	Camera detection ≤ 2 m, line of sight → Pi Camera + MobileNet-SSD person detector	$255(1 - 0.3d/2)$
Blocked	Lidar hits debris in a passage → lidar / ToF	255

Events of the same type within 2 m of a known one are merged. A beacon is also dropped at **doorways** (the robot is between two walls), which are the critical navigation points, and every **3 m** of travel. Each new beacon points `next_id` to the beacon it can hear that is closest to the exit. The beacons therefore form a **tree rooted at the exit**, with extra edges between beacons that hear each other. In the nominal run the Writer covers 100 % of the building in 8.9 min (106 m), finds 5/5 events and drops 22 beacons.

4 Beacon Message and Signal Design

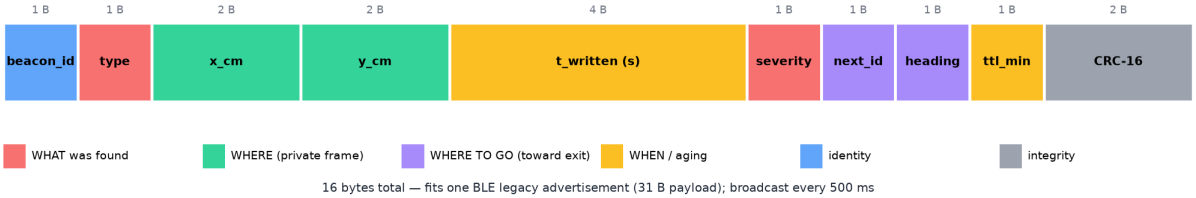


Figure 3: 16-byte beacon packet (little-endian, CRC-16/CCITT). Position is in the Writer's private frame, in cm.

Signal. BLE 5 legacy advertising from an ESP32-C3 on a coin cell. One 16 B packet every 500 ms fits the 31 B advertisement. There is no pairing, so any robot can read it. Range is about 5 m line of sight; RSSI gives proximity (arrival < 0.75 m). A beacon is *write-once*, so it never needs a receiver or a clock. **Message aging** is done by the reader: $c(t) = \frac{\text{sev}}{255} e^{-\text{age}/\tau}$, and $c = 0$ once age $>$ TTL. Stale beacons are still used for navigation, but their events are shown as *unverified*.

Event	τ	TTL	Why
Fire	10 min	30 min	spreads fast
Victim	30 min	90 min	may move
Blocked	2 h	4 h	structural
Waypoint	∞	—	geometry

Table 1: Per-event aging.

In the simulation the Executor logs, for example, "*reads B5: fire written 9.8 min ago, confidence 0.14*". When the uplink is slow (`link` scenario), fire confidence falls below the 0.15 threshold and the commander's risk assessment changes. Aging has a direct effect on decisions.

5 Frame Translation

Frames. The private frame has its origin at the entrance, $+x$ into the building and $+y$ to the left. Both robots start at the same pose, so they share it. The ONA measures the anchor GPS (ϕ_0, λ_0) and the bearing ψ of $+x$ (compass or two GPS fixes along the façade):

$$N = x\cos\psi + y\sin\psi, \quad E = x\sin\psi - y\cos\psi, \quad \phi = \phi_0 + \frac{N}{R}\cdot\frac{180}{\pi}, \quad \lambda = \lambda_0 + \frac{E}{R\cos\phi_0}\cdot\frac{180}{\pi}, \quad R = 6\,378\,137 \text{ m}.$$

Worked example: ϕ_0=36.8130°, λ_0=10.0600°, ψ=90°, (x, y)=(10, 5) m ⇒ N=5 m, E=10 m ⇒ (36.813045°, 10.060112°) (unit-tested).

Gyro-drift loop closure (our contribution). Cheap MEMS gyros drift. We simulate 0.012 °/s (about 6° per 100 m) plus a 1.5 % wheel scale error and noise. The Writer always ends at the entrance, whose true position is (0,0). The ONA searches for the drift rate β (rad/m) such that re-integrating every odometry step rotated by $-\beta \cdot \text{odo}$ closes the loop. It then moves every beacon by the correction at the same odometer reading. The estimate was $\hat{\beta} = 6.08°/100$ m (true value 6.0).

Beacon position error	mean	max
Raw odometry (seed 7)	0.20 m	0.70 m
Gyro loop closure	**0.13 m**	**0.28 m**
5 seeds, raw → corrected	0.25→0.20	0.65→0.34
Linear spreading (5 seeds)	worse: 0.25 → 0.40 mean	

The classic method of spreading the closure error linearly made things *worse*, because heading drift *rotates* the path. Modelling the physical cause fixed it. Each GPS point carries a ± radius, drawn on the live map.

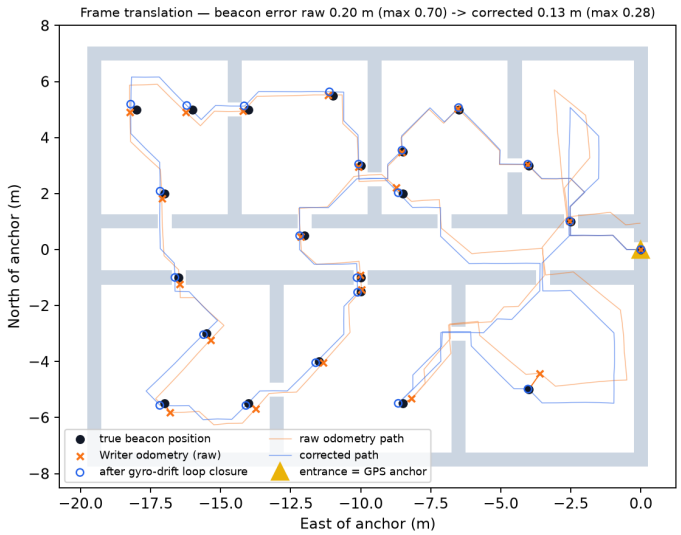


Figure 4: True vs. raw vs. corrected beacon positions (ENU around the anchor), nominal run.

6 Outside Network Area

Role	How (simulation, `sim/ona.py`)	Phase 2 hardware
1 Receive	Binary log dump at the exit (2.4 KB). Each packet is stored *twice*; the CRC picks the valid copy (4 % per-copy corruption injected).	Raspberry Pi 4 Wi-Fi AP; the Writer pushes over TCP when it docks (BLE fallback)
2 Translate	Gyro-drift loop closure → ENU → WGS-84, ± uncertainty; builds the beacon graph	Pi + u-blox NEO-M8N + compass
3 Carry	51-byte LoRa frames (SF7, 869.5 MHz, 10 % duty-cycle sub-band). 18 B per beacon record (lat/lon at 1e-7°), 12 path points per frame. Stop-and-wait ARQ with 8 retries, then store-and-forward.	2×SX1276 (km range line of sight), relay if needed, Iridium SBD fallback
4 Brief	Mission JSON: target, route = Dijkstra over the beacon graph (+40 m penalty near confident fire), beacon table, edges, fire beacons to keep clear of	Same AP, before entry

Results: the nominal map uses **23 frames (844 B)**, delivered in 30 transmissions with 7 losses recovered. At 45 % loss it took 64 transmissions and no frame was lost. The **command post** decodes the frames into a live Leaflet/OpenStreetMap page (`out/live_map.html`) showing events, confidence, age, ± circles and both robot paths. The commander scores victims with score $= \frac{c_{\text{now}}}{1+\text{route}/30} \times (0.3$ if the route passes within 3 m of a confident fire). Only the decision goes back down, and it goes to the ONA.

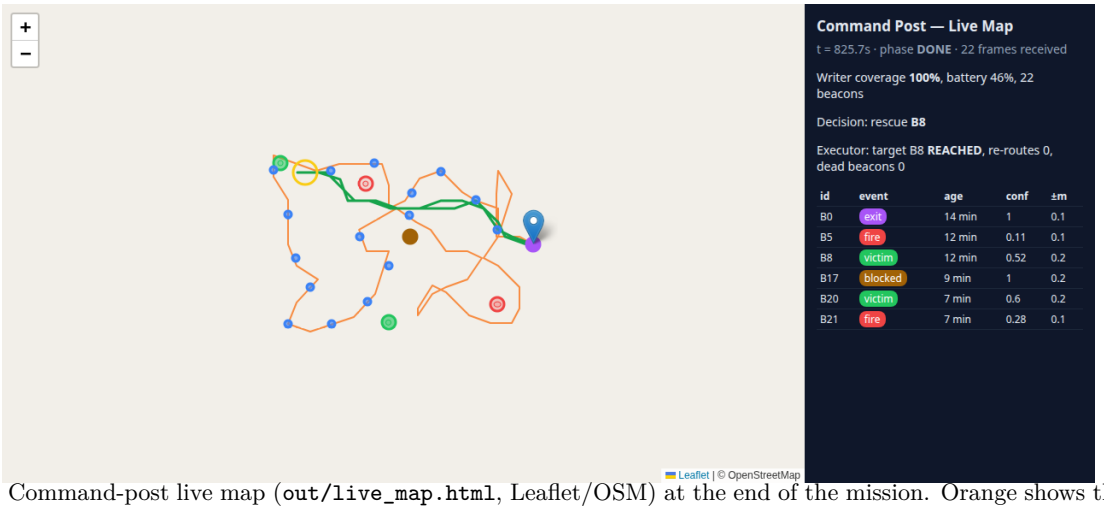


Figure 5: Command-post live map (`out/live_map.html`, Leaflet/OSM) at the end of the mission. Orange shows the Writer's corrected path, green the Executor's path, and the yellow ring the reached victim. The side panel lists each event's age, decayed confidence and ± error.

7 Executor Robot

The Executor receives the brief, has **no map**, and follows the route beacon by beacon. For each beacon it: plans locally with its own lidar (unknown space treated as free); declares *arrival* when the beacon's RSSI shows it is < 0.75 m away; uses **RSSI-gradient search** when it reaches the recorded position without being close; and re-localises when it drives over a beacon. If a beacon is **silent** at its recorded position for 8 steps, it is declared dead and the robot **re-routes** over the graph from a virtual node at its current position. If the local path to the next beacon is $> 3\times$ longer than expected, that **edge is marked blocked** and removed. At the target it walks to the reported event position, **confirms the victim with its camera** (≤ 1 m), drops the aid kit and returns. In **recovery mode** (Writer lost) it explores, reads the Writer's beacons, goes to the first victim beacon it hears, and brings all beacons read back to the ONA.

8 Simulation Demo and Results

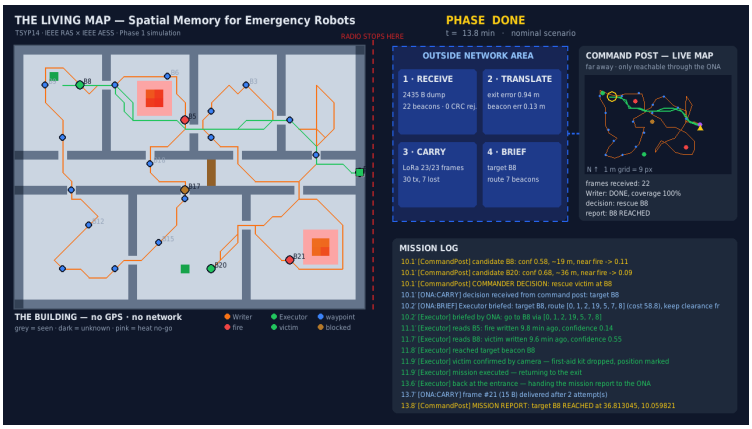


Figure 6: Simulator (Pygame) at the end of the nominal mission: building with the Writer's (orange) and Executor's (green) trails, beacons, the ONA's 4 roles, the command-post live map and the mission log.

Metric (seed 7)	Value
Writer coverage / time	100 % / 8.9 min
Distance / beacons	106 m / 22
Events found	5/5 (2F, 2V, 1 blocked)
Exit (closure) error	0.94 m
Beacon error (corr.)	0.13 m mean
Uplink	23 frames, 844 B, 30 tx
Decision	victim B8 (0.58 conf., 19 m)
Executor	41 m, 3.4 min, confirmed
Whole mission	13.8 min
Tests	21/21 pass

Table 2: Nominal mission results.

Run with `python -m sim.main` (options `-fail beacon|block|writer|link|battery`, `-headless`, `-record`). The 40×30 building (20×15 m) has two fires, two victims and debris that cuts the main corridor, so every route has to pass through rooms. The simulator uses the *same* packet codec, translation, framing and routing code that will run on the ONA Pi and the robots in Phase 2.

9 Failure Cases

Failure	**Detection**	**Response**	**Simulated**
Writer dies inside	No return within 15 min	Executor in recovery mode reads the Writer's beacons	✓ success (61 % explored)
Beacon battery dies	Silent at its position for 8 steps	Mark dead, re-route on the graph	✓ B19 dead → [18,7,8]
Doorway collapses later	Lidar debris, path $> 3\times$ expected	Local detour / remove edge	✓ success
LoRa link degraded (45 %)	Missing ACKs	ARQ, 8 retries, store-and-forward	✓ 64 tx, 0 lost
Low battery	1.3× cost-to-exit rule	Early return, partial map still briefs	✓ 43 % coverage, success
Corrupted packet	CRC-16	Duplicate copy used	✓ always on
Odometry drift	Loop-closure error	Gyro drift removed, ± circles	✓ max 0.70→0.28 m
Stale information	age $>$ TTL / low c	Unverified, lower priority	✓ always on
Wrong anchor ψ	Points outside footprint	Re-measure from 2 GPS fixes	design
No decision received	Downlink timeout	ONA default policy	design

10 Implementation Plan (Phase 2) and Innovation

Weeks (2026)	Milestone
6–19 Oct	Hardware, both chassis, PID, ESP32-C3 beacons
20–26 Oct	Writer odometry + mapping; ONA dump + translation
27 Oct–2 Nov	Frontier exploration, sensors, drop magazine
3–9 Nov	LoRa ONA↔command post, live map, range test
10–16 Nov	Executor beacon navigation + re-routing
17–23 Nov	Integration in a mock building
24–30 Nov	Failure tests, user manual, pitch

Bill of materials: about 1,300–1,600 TND (2 chassis, 2 Pi 4, ESP32-S3, ToF/lidar, IMUs, thermal, camera, 12 beacons, 2 LoRa, GPS). Details in `docs/implementation_plan.md`.

Innovation

- Knowledge **outlives the robot**: battery-aware return plus write-once beacons, shown working when the Writer dies.
- Beacons form a **navigation graph** (`next_id` tree + neighbour edges), not isolated markers.
- **Gyro-drift loop closure**, which halves the maximum error; the naïve method fails.
- **Per-event aging** that changes the commander's decisions.
- Double-copy dump, CRC and LoRa store-and-forward, so **nothing is silently lost**.